

RAG to Agentic RAG

A Comprehensive Course — From Foundations to Production
Agentic Systems

15 In-Depth Chapters

Feynman
Explanations

Real-World Examples

Production Best
Practices

Ch 01 Introduction to RAG

Ch 02 Evolution of Retrieval

Ch 03 Data Ingestion

Ch 04 Embeddings

Ch 05 Vector Databases & Indexing

Ch 06 Retrieval Techniques

Ch 07 Retrieval Evaluation

Ch 08 Re-ranking

Ch 09 Augmentation

Ch 10 Generation

Ch 11 Generation Evaluation

Ch 12 RAG vs Fine-tuning

Ch 13 RAG with Tools

Ch 14 Agentic RAG

Ch 15 Production Best Practices

Introduction to RAG

Retrieval-Augmented Generation (RAG) is one of the most transformative patterns in modern AI engineering. It bridges the gap between the remarkable language capabilities of Large Language Models (LLMs) and the ever-growing, ever-changing world of factual knowledge. This chapter establishes the foundational reasoning behind RAG — why it exists, what problems it solves, and how it works at a high level.

1.1 Why Do Large Language Models Need RAG?

Large Language Models like GPT-4, Claude, and Llama are trained on massive corpora of text data. During training, these models compress billions of tokens of information into their neural network weights — a form of **parametric memory**. This is impressive, but it comes with fundamental limitations.

LLMs are essentially very sophisticated pattern-matching engines. They learn statistical relationships between tokens, phrases, and concepts. When you ask an LLM a factual question, it reconstructs an answer from these learned patterns. The problem is that this reconstruction is lossy, static, and can produce confident-sounding but incorrect outputs — a phenomenon known as *hallucination*.

RAG solves this by giving the model a dynamic, external knowledge source to consult at inference time. Instead of relying solely on what the model memorized during training, RAG retrieves relevant documents from a knowledge base and feeds them into the model's context window before generating an answer. This makes the model's output grounded in verifiable, up-to-date information.

Core Motivations for RAG

- **Factual grounding:** Retrieved documents act as a source of truth, reducing hallucinations.
- **Up-to-date knowledge:** The knowledge base can be updated without retraining the model.
- **Domain specificity:** Organizations can index proprietary documents the model was never trained on.
- **Transparency:** Retrieved sources can be cited, making the system auditable.
- **Cost efficiency:** Updating a vector database is far cheaper than fine-tuning a model.

1.2 Context Window Limitations

Every LLM has a finite **context window** — the maximum number of tokens it can process at once. Early models like GPT-2 had context windows of just 1,024 tokens. Modern models have expanded dramatically: GPT-4 Turbo supports 128K tokens, Claude 3 supports 200K tokens, and Gemini 1.5 Pro has demonstrated 1M+ token contexts. Despite this progress, context window limits remain a

practical engineering constraint for most production systems.

Even with large context windows, simply stuffing all available documents into the context is not a viable strategy. First, it is expensive: LLM pricing is typically per token, so a 200K-token context window filled on every query would cost dramatically more than retrieving 5–10 relevant chunks. Second, research shows that LLMs struggle with the **'lost in the middle'** problem — they tend to attend well to information at the beginning and end of the context, but poorly to information buried in the middle. Third, latency scales with context length, hurting user experience.

RAG addresses these limitations by performing **targeted retrieval**: instead of reading an entire knowledge base, the system retrieves only the most relevant pieces of information for each specific query. This keeps the context window lean, costs manageable, and generation quality high.

Approach	Context Cost	Knowledge Freshness	Scalability	Hallucination Risk
No RAG (parametric only)	Low	Frozen at training cutoff	High	High
Full-context stuffing	Very High	Current	Poor	Low (but costly)
RAG (targeted retrieval)	Low-Medium	Current	High	Low

Table 1.1: Comparing knowledge access strategies for LLMs

1.3 Knowledge Cutoff and Hallucinations

Every LLM has a **training data cutoff** — a date after which no new information was incorporated into its weights. For example, a model trained through mid-2024 will be unaware of events, research papers, product launches, or regulatory changes that occurred after that date. In fast-moving domains like finance, law, medicine, and technology, this cutoff quickly becomes a serious liability.

Hallucinations arise when a model is asked about something outside its training distribution or beyond its training cutoff. The model, trained to always produce fluent, confident-sounding text, will generate plausible-sounding but factually incorrect answers. Hallucinations can be subtle — a wrong date, a misattributed quote, or a slightly incorrect drug dosage — making them particularly dangerous in high-stakes applications.

RAG mitigates both problems simultaneously. The knowledge base can be continuously updated with new information, eliminating the staleness problem. And by grounding generation in retrieved documents, RAG gives the model factual anchors that constrain its outputs to verifiable information. The model is essentially told: 'Answer using these documents. If you can't find the answer here, say so.'

Types of Hallucinations in LLMs

- **Factual hallucination:** Stating incorrect facts (wrong dates, wrong people, wrong statistics).
- **Fabricated citations:** Inventing academic papers, articles, or books that don't exist.
- **Temporal hallucination:** Presenting outdated information as current.
- **Logical hallucination:** Drawing incorrect conclusions from correct premises.
- **Confabulation:** Filling gaps in knowledge with plausible-sounding but invented details.

1.4 High-Level RAG Architecture

At its core, a RAG system has two major pipelines that operate at different times: the **indexing pipeline** (offline) and the **retrieval-augmented generation pipeline** (online, at inference time).

The Indexing Pipeline (Offline)

This pipeline prepares the knowledge base for retrieval. It runs once and is refreshed periodically when new documents are available.

- **Step 1 — Data Collection:** Raw documents (PDFs, web pages, database records, etc.) are gathered from source systems.
- **Step 2 — Preprocessing:** Documents are cleaned, parsed into plain text, and normalized.
- **Step 3 — Chunking:** Documents are split into smaller, semantically meaningful pieces (chunks).
- **Step 4 — Embedding:** Each chunk is converted into a dense vector representation using an embedding model.
- **Step 5 — Indexing:** Vectors (with associated metadata and text) are stored in a vector database.

The Inference Pipeline (Online, per Query)

- **Step 1 — Query Reception:** A user submits a natural-language query.
- **Step 2 — Query Embedding:** The query is embedded into the same vector space as the documents.
- **Step 3 — Retrieval:** The vector database finds the most semantically similar document chunks.
- **Step 4 — Augmentation:** Retrieved chunks are formatted and injected into a prompt template.
- **Step 5 — Generation:** The LLM generates an answer grounded in the retrieved context.
- **Step 6 — Response:** The answer (with optional source citations) is returned to the user.

FEYNMAN TECHNIQUE — Retrieval-Augmented Generation

Explain it Simply: RAG is like taking an open-book exam instead of a closed-book one. Without RAG, an AI has to rely entirely on what it memorized during training — like a student taking a test from memory, risking wrong answers. With RAG, the AI can look things up in a reference book first, then write its answer based on what it actually found — more accurate, more current, more reliable.

Real-World Analogy: Think of it like a brilliant research assistant. Without RAG, the assistant answers from memory alone — fast, but sometimes wrong. With RAG, the assistant first searches a filing cabinet full of up-to-date documents, pulls out the most relevant ones, reads them, and then formulates a response. The filing cabinet is your vector database; pulling out relevant documents is retrieval; writing the response is generation.

Concrete Example: You build a customer support chatbot for a software product. The product updates every month. Without RAG, the LLM only knows about features that existed before its training cutoff. With RAG, you index your latest documentation, and when a user asks 'How do I use the new batch export feature?', the system retrieves the relevant documentation page and the LLM answers accurately from it, even if the feature was released last week.

Key Takeaways — Chapter 1

- LLMs rely on parametric memory that is static, finite, and prone to hallucinations.
- Context window constraints make naive document stuffing impractical at scale.
- RAG combines the language understanding of LLMs with dynamic, updatable external knowledge.
- The RAG pipeline has two phases: an offline indexing phase and an online inference phase.
- RAG is not a replacement for LLMs but a powerful architectural pattern that makes them more reliable.

Evolution of Retrieval

Before RAG can be fully understood, we must appreciate the decades of research and engineering that went into information retrieval (IR). Modern semantic search and vector retrieval did not appear overnight — they are the culmination of 70+ years of effort to answer a deceptively simple question: how do you find relevant information in a large collection of documents?

2.1 History of Information Retrieval

Information Retrieval as a formal discipline emerged in the 1950s and 1960s, driven by the need to organize and search scientific literature. Gerard Salton's SMART system (1960s) introduced the **Vector Space Model**, which represented documents and queries as vectors over a vocabulary, computing relevance through vector similarity. This was a foundational insight that directly presages modern embedding-based retrieval.

The 1970s and 1980s saw the development of probabilistic retrieval models, most notably the **Binary Independence Model** and the **Robertson-Sparck Jones framework** that would eventually lead to BM25. These models brought statistical rigor to relevance ranking, moving beyond simple keyword matching toward a probabilistic understanding of what makes a document relevant.

The 1990s brought the World Wide Web, which created an unprecedented scale challenge. Search engines like AltaVista and early Google had to retrieve from billions of documents. Google's PageRank introduced **link-based authority signals**, demonstrating that retrieval quality improves dramatically when you combine content matching with structural signals from the document graph.

2.2 Traditional Search Systems

Traditional search systems are built on **inverted indices** — a data structure that maps each unique term to the list of documents containing it, along with positions and frequency counts. Given a query 'climate change solutions', an inverted index lookup finds all documents containing 'climate', 'change', or 'solutions', then ranks them by a relevance formula.

TF-IDF: The Workhorse of Classical IR

TF-IDF (Term Frequency-Inverse Document Frequency) is the most influential classical relevance formula. It scores each term in a document as the product of two factors: how often the term appears in this document (TF) and how rare the term is across all documents (IDF). Rare terms that appear frequently in a document are strong relevance signals; common words like 'the' and 'is' are down-weighted by the IDF factor.

TF-IDF works well for exact keyword matching but has significant limitations. It treats each word independently (bag-of-words assumption), ignoring word order and semantic relationships. A search

for 'car' won't retrieve documents about 'automobile', even if they are perfectly relevant.

BM25: Best Match 25

BM25 (Okapi Best Match 25) is the gold standard of classical keyword retrieval, developed in the 1990s by Robertson and colleagues. It refines TF-IDF with two key improvements: **term frequency saturation** (adding the same term 100 times shouldn't increase relevance 100x) and **document length normalization** (longer documents have more terms by chance alone, so they shouldn't be unfairly penalized). BM25 remains competitive with neural approaches for many retrieval tasks and is still widely used as the first-stage retriever in modern systems.

2.3 From Keyword Search to Semantic Retrieval

The fundamental limitation of keyword-based retrieval is its reliance on **lexical overlap** — documents are only relevant if they share words with the query. This fails to capture synonymy (car vs. automobile), polysemy (bank: financial vs. riverbank), and conceptual similarity (query: 'how to lose weight' matching 'diet and exercise programs').

Latent Semantic Analysis (LSA) in the early 1990s was the first major attempt to capture semantic relationships. LSA applies Singular Value Decomposition to the term-document matrix, discovering latent topics that group semantically related terms. While promising, it was computationally expensive and difficult to update incrementally.

The real breakthrough came with **neural embeddings**. Word2Vec (2013) showed that neural networks trained on word co-occurrence learn vector representations where geometric distance encodes semantic similarity — the famous example: 'king' - 'man' + 'woman' $\approx$ 'queen'. Sentence-BERT (2019) extended this to full sentences, producing fixed-size embeddings where semantically similar sentences are close in vector space. This enabled practical semantic search.

Era	Key Technology	Approach	Strength	Limitation
1960s–80s	Vector Space / TF-IDF	Term frequency weighting	Simple, efficient	No semantics
1990s	BM25, Boolean IR	Probabilistic ranking	Robust, interpretable	Lexical only
1990s–00s	LSA / LDA	Topic modelling	Captures topics	Slow, static
2013–17	Word2Vec, GloVe	Word embeddings	Word semantics	No sentence-level
2018–19	BERT, Sentence-BERT	Contextual embeddings	Sentence semantics	High latency
2020+	Bi-encoders, ColBERT	Dense retrieval	Fast + semantic	Requires indexing

Table 2.1: Key milestones in information retrieval

2.4 Why Retrieval Remains the Foundation of RAG

Even as LLMs have grown in capability, retrieval remains the load-bearing foundation of any RAG system. The quality of retrieval directly caps the quality of generation: if the retriever fails to surface the relevant document, even the most capable LLM cannot generate a correct answer.

This insight motivates what practitioners call the **retrieval bottleneck**: in end-to-end RAG evaluation, retrieval failures account for the majority of system failures. An LLM can handle noisy or partially relevant context gracefully, but it cannot hallucinate its way to a correct answer about a document it never saw. Investing in retrieval quality — better embedding models, smarter chunking, hybrid retrieval — typically yields larger quality improvements than upgrading the generation model.

FEYNMAN TECHNIQUE — The Evolution of Search

Explain it Simply: Search has evolved from a simple keyword lookup to a sophisticated understanding of meaning. Early search engines were like a librarian who could only find books containing your exact words. Modern semantic search is like a librarian who understands what you're looking for even if you don't use the exact terminology — you can ask for 'something about cars' and they'll bring you books about automobiles, vehicles, and transportation.

Real-World Analogy: Imagine you're at a library. An old card catalog system (TF-IDF) can only find books that literally contain the word 'automobile.' A smart modern librarian (semantic search) knows that 'automobile,' 'car,' 'vehicle,' and 'motor transport' all mean the same thing and can recommend books about any of them when you ask about 'cars.' The librarian also knows the context — whether you want engineering books, history books, or travel books about cars.

Concrete Example: Query: 'cardiac arrest symptoms'. An old keyword system fails to match a document that talks about 'heart attack warning signs' — same concept, different words. A semantic retrieval system using BERT-style embeddings places both queries in similar vector positions, successfully retrieving the relevant medical document even with zero keyword overlap.

Key Takeaways — Chapter 2

- Information Retrieval has 70+ years of history, culminating in modern semantic search.
- TF-IDF and BM25 are powerful lexical retrieval methods still widely used today.
- Neural embeddings (Word2Vec, BERT, Sentence-BERT) enabled true semantic retrieval.
- Retrieval quality is the single most impactful factor in RAG system performance.

- Modern RAG systems often combine lexical (BM25) and semantic (dense) retrieval — called hybrid retrieval.

Data Ingestion

Data ingestion is the unglamorous but critical first step in building any RAG system. The quality of your retrieval is bounded by the quality of your ingested data. Garbage in, garbage out — if your chunking strategy is poor, your metadata is missing, or your preprocessing introduces artifacts, even the best retrieval model will struggle. This chapter covers the complete data ingestion pipeline from raw documents to indexed, searchable knowledge.

3.1 Preparing Documents for Retrieval

The first challenge is handling the diversity of document formats that exist in real-world knowledge bases. Enterprise data rarely arrives in a clean, uniform format — it comes as PDFs with complex layouts, HTML web pages with navigation noise, Microsoft Word documents, PowerPoint presentations, Markdown files, database records, and API responses.

Each format requires a different parsing strategy. PDFs are particularly challenging because they describe visual layout rather than semantic structure — text extraction can fail for scanned documents (requiring OCR), multi-column layouts can produce garbled reading order, and tables often become corrupted text strings.

Common Document Formats and Parsing Approaches

- **PDF:** Use libraries like PyMuPDF, pdfplumber, or LlamaParse for structure-aware extraction.
- **HTML/Web:** Use BeautifulSoup or Trafilatura to extract main content, stripping navigation/ads.
- **DOCX/PPTX:** Use python-docx or python-pptx with structural metadata preservation.
- **Markdown:** Parse headers and sections to preserve document hierarchy.
- **CSV/Excel:** Convert structured data to text with column context preservation.
- **Images:** Apply OCR (Tesseract, AWS Textract) for text extraction; use VLMs for image understanding.

3.2 Data Cleaning and Preprocessing

Raw extracted text from real-world documents is often noisy. Cleaning it before indexing dramatically improves retrieval quality. Common preprocessing steps include:

- **Whitespace normalization:** Collapse multiple spaces, normalize newlines, remove form feeds.
- **Encoding handling:** Ensure consistent UTF-8 encoding; handle special characters.
- **Boilerplate removal:** Strip page headers, footers, page numbers, and copyright notices.
- **Language detection:** Identify document language for multilingual corpora.

- **Deduplication:** Identify and remove near-duplicate documents using MinHash or embedding similarity.
- **PII scrubbing:** Remove personal identifiable information where required by compliance.
- **Quality filtering:** Remove very short, very long, or uninformative text segments.

3.3 Chunking Strategies

Chunking is the process of splitting documents into smaller pieces for embedding and retrieval. This is one of the most critical design decisions in a RAG system. A chunk that is too small lacks context; a chunk that is too large reduces retrieval precision and risks hitting the embedding model's token limit.

Fixed-Size Chunking

The simplest approach: split documents into chunks of N tokens with an overlap of M tokens. For example, chunks of 512 tokens with 64 tokens of overlap. The overlap ensures that sentences split at chunk boundaries are still represented in adjacent chunks.

- **Pros:** Simple to implement, predictable chunk sizes, fast.
- **Cons:** Splits in the middle of sentences or paragraphs, losing semantic coherence.

Recursive Character Splitting

A smarter fixed-size approach used by LangChain's RecursiveCharacterTextSplitter. It tries to split on paragraph boundaries first (`\n\n`), then sentence boundaries (`\n`), then word boundaries (`' '`), falling back to character-level only when necessary. This preserves semantic coherence far better than naive fixed-size splitting.

Semantic Chunking

Group sentences into chunks based on embedding similarity. Compute embeddings for each sentence, then split when the semantic distance between adjacent sentences exceeds a threshold. This produces chunks where all sentences are topically cohesive. More expensive but produces the highest quality chunks.

Document Structure-Aware Chunking

Use the document's own structure (headers, sections, paragraphs) as natural chunk boundaries. A PDF section becomes one chunk. A Markdown heading-level section becomes one chunk. This preserves the author's intended information units.

Parent Document Retrieval

A two-level approach: index small chunks for precise retrieval, but when a chunk is retrieved, return its parent (larger document section) for richer context. Balances retrieval precision with generation context quality.

Strategy	Chunk Quality	Semantic Coherence	Complexity	Best For
Fixed-size	Low-Medium	Low	Very Low	Quick prototypes
Recursive character	Medium	Medium	Low	General-purpose
Sentence-based	Medium-High	High	Medium	Narrative documents
Semantic chunking	High	Very High	High	Quality-critical systems
Structure-aware	High	High	Medium-High	Structured documents (PDFs, HTML)
Parent document	High	High	High	Long documents with hierarchy

Table 3.1: Chunking strategy comparison

3.4 Metadata Enrichment

Every chunk should be stored with rich metadata that enables filtered retrieval. Metadata transforms a vector database from a pure semantic search engine into a hybrid system that can combine meaning with structured filtering.

- **Source metadata:** document filename, URL, database ID, file path.
- **Temporal metadata:** creation date, last modified date, publication date.
- **Structural metadata:** page number, section title, heading hierarchy, chunk index.
- **Semantic metadata:** automatically extracted topics, entities, keywords using NLP.
- **Access metadata:** department, classification level, access permissions.
- **Quality metadata:** confidence score from OCR, extraction quality flags.

3.5 Document Parsing Pipelines

A production data ingestion pipeline orchestrates all the above steps in a reliable, scalable, and observable manner. Key frameworks and tools include:

- **Unstructured.io:** Open-source library handling 20+ document types with layout detection.
- **LlamaParse:** LlamaIndex's proprietary parser with excellent PDF table and image handling.
- **LangChain document loaders:** Large library of format-specific loaders (100+ formats).
- **Apache Tika:** Battle-tested Java-based parser for enterprise document processing.
- **Azure AI Document Intelligence:** Cloud service with exceptional PDF layout understanding.

FEYNMAN TECHNIQUE — Document Chunking

Explain it Simply: Chunking is like making index cards from a textbook. If you want to look up a specific fact later, you don't search through the entire textbook — you search through your index cards. But if your index cards are too small (just one word each), they lose context. If they're too big (one card per chapter), they're hard to compare. The sweet spot is a card that captures one complete idea — a paragraph or a few related sentences.

Real-World Analogy: Think of a cookbook. If you make index cards for it, you wouldn't put one card per ingredient or one card per entire chapter. You'd make one card per recipe — just the right size to capture a complete, self-contained idea. Chunking documents works the same way: find the natural boundaries of complete ideas and make those your searchable units.

Concrete Example: You have a 200-page technical manual. Using fixed-size chunking (512 tokens, 64 overlap), you might split a procedure in the middle: chunk A ends with 'insert the cable' and chunk B starts with 'into the port'. If a user asks about cable insertion, they retrieve chunk A but miss the completion. With structure-aware chunking, the entire procedure becomes one chunk, and retrieval returns the complete, actionable information.

Key Takeaways — Chapter 3

- Data quality directly bounds retrieval quality — preprocessing is non-negotiable.
- Chunking strategy is one of the highest-leverage design decisions in a RAG system.
- Semantic and structure-aware chunking outperform simple fixed-size approaches for quality.
- Metadata enrichment enables hybrid (semantic + filtered) retrieval at query time.
- Production pipelines need to handle 20+ document formats reliably at scale.

Embeddings

Embeddings are the mathematical heart of modern semantic search. They are the mechanism by which language — inherently symbolic and discrete — is transformed into a continuous geometric space where similarity of meaning corresponds to proximity in space. Understanding embeddings deeply is essential for making intelligent choices about which models to use, how to evaluate them, and how to optimize them for your domain.

4.1 Why Text Needs to Be Converted into Vectors

Computers cannot directly compare the meaning of two sentences. 'The patient presented with chest pain' and 'The individual experienced angina' are semantically identical, but a computer treating them as strings of characters sees zero similarity. To enable semantic comparison, we need to map text into a mathematical space where geometric operations (like computing distance or similarity) correspond to semantic operations (like measuring meaning similarity).

An **embedding** is a function that maps a piece of text — a word, sentence, or document — to a fixed-length vector of real numbers (e.g., 384, 768, or 1536 dimensions). Well-trained embeddings exhibit a remarkable property: texts with similar meaning produce vectors that are geometrically close. This allows us to build approximate nearest neighbor indices that retrieve semantically similar texts in milliseconds, even over millions of documents.

The Geometry of Meaning

- **Cosine similarity:** Measures the angle between two vectors; range $[-1, 1]$; 1 = identical direction, 0 = orthogonal.
- **Dot product:** Related to cosine similarity but also sensitive to vector magnitude; used with normalized vectors.
- **Euclidean distance:** Straight-line distance; inversely related to similarity; less common in practice.
- **Semantic clusters:** Vectors of related concepts cluster together; e.g., medical terms form a cluster distant from legal terms.

4.2 Sparse (Keyword-Based) Embeddings

Sparse embeddings represent text as vectors over the entire vocabulary, with most dimensions being zero. Each non-zero dimension corresponds to a term that appears in the text.

TF-IDF Vectors

The classical sparse embedding. Each dimension is a vocabulary term; the value is the TF-IDF score. With a vocabulary of 100,000 terms, each document produces a vector of 100,000 dimensions, but typically only 50–500 are non-zero. Sparse vectors have excellent interpretability (each non-zero dimension corresponds to a meaningful word) but no semantic awareness.

SPLADE (Sparse Lexical and Expansion Model)

SPLADE is a neural approach to sparse representations. Trained using a masked language model objective, it learns to assign weights not just to terms that appear in the text, but also to **semantically related terms** not present in the text. A SPLADE embedding for 'automobile' might have non-zero weights for 'car', 'vehicle', 'transport' — achieving semantic generalization while maintaining sparsity. SPLADE achieves state-of-the-art performance in keyword-focused retrieval tasks.

4.3 Dense (Vector-Based) Embeddings

Dense embeddings are fixed-size vectors where all (or most) dimensions have non-zero values. Unlike sparse embeddings that map to vocabulary terms, dense embedding dimensions have no direct human-interpretable meaning — they are learned distributed representations that encode semantic information across all dimensions simultaneously.

Bi-Encoder Architecture

The dominant architecture for retrieval-focused embeddings. A pre-trained transformer (typically BERT or its variants) encodes a text passage, and the [CLS] token embedding (or the mean-pooled output) becomes the passage embedding. Queries and documents are encoded **independently** using the same encoder, enabling pre-computation of document embeddings. At query time, only the query needs to be encoded, making bi-encoders extremely fast.

Key Dense Embedding Models

Model	Dims	Max Tokens	Strengths	Use Case
text-embedding-3-small (OpenAI)	1536	8192	High quality, fast, cheap	General RAG
text-embedding-3-large (OpenAI)	3072	8192	Best OpenAI quality	Quality-critical
BAAI/bge-large-en-v1.5	1024	512	Open-source SOTA	On-prem / open
sentence-transformers/all-MiniLM-L6	384	256	Fast, lightweight	Edge / low-resource
intfloat/e5-large-v2	1024	512	Strong multilingual	Multilingual
Cohere embed-v3.0	1024	512	Excellent quality	Production SaaS

nomic-embed-text-v1.5	768	8192	Long context, open	Long documents
-----------------------	-----	------	--------------------	----------------

Table 4.1: Popular embedding models comparison

4.4 Choosing the Right Embedding Model

Model selection depends on multiple factors. There is no single best embedding model — the optimal choice depends on your domain, language, infrastructure constraints, and quality requirements.

- **Domain alignment:** General-purpose models may underperform on specialized vocabulary (medical, legal, code). Consider domain-specific fine-tuning.
- **Language:** For multilingual content, use models like mE5 or Cohere multilingual. English-only models fail on other languages.
- **Context length:** Choose a model whose max token length comfortably accommodates your chunk size.
- **Latency budget:** Smaller models (384d, 768d) embed faster; larger models (1536d+) are slower but higher quality.
- **Cost:** Cloud APIs charge per token. High query volumes may justify switching to open-source self-hosted models.
- **Normalization:** Check if the model outputs normalized vectors (L2 norm = 1); required for efficient dot-product similarity.

4.5 Embedding Quality Evaluation

Evaluating embedding models rigorously before deployment prevents costly mistakes. The standard benchmark is **MTEB (Massive Text Embedding Benchmark)**, which evaluates models across 58 tasks in 112 languages. However, always evaluate on your specific domain and query types — MTEB scores can be misleading for specialized applications.

- **Retrieval@K:** Given test queries with known relevant documents, measure what fraction of relevant docs appear in top-K results.
- **MRR (Mean Reciprocal Rank):** Average of the reciprocal rank of the first relevant result.
- **NDCG@K:** Normalized Discounted Cumulative Gain — accounts for result ranking order.
- **Domain-specific test sets:** Create a small gold-standard evaluation set from your actual documents and queries.

FEYNMAN TECHNIQUE — Embeddings

Explain it Simply: An embedding is like a GPS coordinate for meaning. Just as GPS coordinates let you measure how close two physical locations are, embeddings let you measure how close two pieces of text are in meaning. Texts with similar meanings get coordinates that are close together; texts about completely different topics get coordinates far apart.

Real-World Analogy: Imagine a city map where every restaurant is placed based on cuisine similarity rather than physical location: all Italian restaurants cluster together, Indian restaurants form another cluster, and they're far from the Japanese restaurant cluster. Now, when you want 'something like carbonara', you look at the Italian food neighborhood and find everything nearby — even if some dishes don't have 'carbonara' in their name.

Concrete Example: You embed 'patient presented with chest pain' and 'angina attack symptoms'. A keyword search finds zero overlap. But their embedding vectors have a cosine similarity of 0.91, placing them very close in the medical knowledge embedding space. A RAG system using embeddings will successfully retrieve documents about 'angina' when a user asks about 'chest pain' — a critical capability in clinical applications where exact terminology varies widely.

Key Takeaways — Chapter 4

- Embeddings map text to vectors where geometric proximity = semantic similarity.
- Sparse embeddings (BM25, SPLADE) excel at keyword precision; dense embeddings capture meaning.
- Bi-encoder models encode queries and documents independently — enabling fast pre-computation.
- Model selection depends on domain, language, latency, and cost constraints.
- Always evaluate embeddings on domain-specific test sets, not just MTEB leaderboard scores.

Vector Databases & Indexing

Vector databases are the storage and retrieval backbone of any RAG system. They are purpose-built to store millions or billions of high-dimensional vectors and answer nearest-neighbor queries in milliseconds. Understanding how they work internally — and how to choose and configure them — is essential for building fast, scalable, and accurate retrieval systems.

5.1 Why Vector Databases Are Required

A naive approach to vector retrieval would be to compute the cosine similarity between the query vector and every vector in the database, then sort and return the top-K. This is called **exact k-NN (k-nearest neighbor) search**. For small collections (< 100K vectors), it works fine. But at scale — 10M vectors of 768 dimensions — a brute-force linear scan takes seconds per query, which is completely unacceptable for real-time applications.

Vector databases solve this with **Approximate Nearest Neighbor (ANN) algorithms** that trade a small amount of accuracy for dramatic speed gains. They can answer queries over 100M+ vectors in under 10 milliseconds, with >95% recall compared to exact search. Beyond raw vector storage and retrieval, modern vector databases also provide metadata filtering, multi-vector indexing, payload storage, access control, real-time upserts, and horizontal scaling — capabilities that a traditional database or a NumPy array cannot match.

5.2 Vector Storage Fundamentals

A vector database stores each document chunk as a **record** consisting of:

- **Vector (embedding):** The high-dimensional float array produced by the embedding model.
- **Payload/Metadata:** Structured attributes (source, date, section, access level) for filtering.
- **Document text:** The original chunk text, returned alongside search results.
- **ID:** A unique identifier for the record.

The critical design choice is the **indexing structure** — how vectors are organized to enable fast approximate nearest-neighbor search. Different index types offer different tradeoffs between query speed, index build time, memory consumption, and recall accuracy.

5.3 Popular Vector Databases

Database	Type	Strengths	Best For
Qdrant	Open-source / Cloud	Fast HNSW, rich filtering, Rust-based, low memory	Production RAG, on-prem

Pinecone	Managed Cloud	Zero-ops, scalable, serverless option	Startups, rapid deployment
Weaviate	Open-source / Cloud	GraphQL API, multi-modal, modules ecosystem	Multi-modal RAG
Chroma	Open-source	Simple API, embeddable, great for development	Development, small projects
Milvus	Open-source / Cloud	Massive scale (billion vectors), distributed	Enterprise, huge scale
pgvector	PostgreSQL ext.	Reuses existing Postgres infra, simple ops	Existing Postgres shops
FAISS (Meta)	Library	State-of-the-art algorithms, GPU support	Research, custom systems

Table 5.1: Popular vector databases comparison

5.4 Vector Indexing Techniques

Flat Index (Exact Search)

No indexing — every query performs a linear scan over all vectors. Exact recall but $O(N)$ query time. Practical only for $N < 100K$ or in GPU-accelerated settings.

IVF (Inverted File Index)

Clusters vectors into N_{list} Voronoi cells using K-means. At query time, only the closest n_{probe} cells are searched. Fast at the cost of recall (the relevant vector might be in an unprobed cell). Good for large datasets when memory is constrained.

HNSW (Hierarchical Navigable Small World)

HNSW is the dominant ANN index for production vector databases. It builds a multi-layer graph where each node (vector) is connected to its approximate nearest neighbors. The layers form a hierarchy: the top layer has few nodes with long-range connections; the bottom layer has all nodes with short-range connections. Queries start at the top (fast coarse navigation) and drill down to the bottom (fine-grained search).

HNSW's key parameters: **M** (number of bidirectional edges per node, typically 16-64) and **ef_construction** (beam width during index build, typically 100-200). Higher M and ef_construction improve recall but increase memory and build time. At query time, **ef** controls the search beam width — larger ef yields higher recall at the cost of latency.

Product Quantization (PQ)

Compresses vectors by splitting each vector into M sub-vectors and quantizing each to one of k_{sub} centroids. Dramatically reduces memory (often 4-16x) at the cost of some recall. Typically combined with IVF (IVF-PQ) for memory-efficient search over billions of vectors.

5.5 Approximate Nearest Neighbor (ANN) Search

ANN search finds vectors that are *approximately* closest to the query — not necessarily the exact nearest neighbors, but close enough for practical retrieval purposes. The tradeoff is characterized by two metrics:

- **Recall@K:** What fraction of the true top-K nearest neighbors does the ANN search return? (Target: >95%)
- **Queries Per Second (QPS):** How many queries can the system handle per second?

The ANN recall-speed tradeoff is the fundamental engineering tension in vector retrieval. Systems like HNSW achieve >99% recall at speeds 100-1000x faster than brute-force, making them ideal for production RAG.

FEYNMAN TECHNIQUE — Vector Indexing and HNSW

Explain it Simply: A vector database with HNSW indexing is like a city organized for GPS navigation. Without indexing, finding the nearest coffee shop means checking every location in the city one by one. With HNSW, it's like a navigation system that first zooms out to find your general neighborhood, then zooms in to find the specific block, then checks the nearby streets — finding the answer in milliseconds instead of minutes.

Real-World Analogy: Imagine a library organized not alphabetically, but by how similar books are to each other — books about similar topics are physically close. HNSW is like a map of this library with multiple zoom levels. At the highest zoom level, you can jump between major sections quickly (Science, History, Fiction). At the middle level, you navigate between sub-topics. At the lowest level, you find the exact shelf. Each zoom-in step reduces your search area dramatically.

Concrete Example: You store 5 million product descriptions as 768-dimensional vectors. Without HNSW, finding the 10 most similar products to a query requires 5M cosine similarity calculations (~500ms). With HNSW ($M=32$, $ef=100$), the same query checks ~3,000 candidates and returns in 2ms with 98% recall — making real-time product search feasible.

Key Takeaways — Chapter 5

- Vector databases enable fast (< 10ms) nearest-neighbor search over millions of vectors.
- HNSW is the dominant production indexing algorithm: high recall + high speed.

- Key parameters — `M`, `ef_construction`, `ef` — control the recall-speed-memory tradeoff.
- Qdrant, Pinecone, and Weaviate are leading production vector databases with different tradeoffs.
- Metadata filtering combined with vector search enables powerful hybrid retrieval pipelines.

Retrieval Techniques

Having built our indexed knowledge base, we now need to query it effectively. Different retrieval strategies have different strengths: keyword-based methods excel at exact term matching, semantic methods excel at conceptual similarity, and hybrid methods combine the best of both. Advanced techniques like query expansion and HyDE push retrieval quality even further.

6.1 Keyword-Based Retrieval

Keyword retrieval uses an inverted index to find documents containing terms from the query. The dominant algorithm is **BM25**, which scores documents based on term frequency, inverse document frequency, and document length normalization. BM25 is fast (sub-millisecond for most queries), interpretable, and highly effective for queries where exact terms matter.

BM25 excels at: precise entity names, technical terms, product SKUs, legal citations, and any query where the user knows the exact terminology. It fails at: synonyms, paraphrasing, questions about concepts (not specific terms), and multilingual retrieval.

6.2 Semantic (Vector) Retrieval

Semantic retrieval embeds both the query and documents into a shared vector space and retrieves documents whose embeddings are nearest to the query embedding. This captures conceptual similarity regardless of surface lexical form.

Semantic retrieval excels at: synonym handling, paraphrasing, question-answer matching, and cross-lingual retrieval. It can struggle with: exact entity matching (a subtle change in a model number may not change the embedding much), rare specialized terms (not well-represented in the embedding model's training data), and queries that require precise keyword matching.

6.3 Hybrid Retrieval

Hybrid retrieval combines keyword and semantic retrieval to capture the complementary strengths of both. The standard approach is to run BM25 and vector search in parallel, then merge the result lists using a rank fusion algorithm.

Reciprocal Rank Fusion (RRF)

RRF is the most widely used rank fusion algorithm. For each document, it computes a score based on its rank in each retrieval method's result list, using the formula: **RRF_score = $\sum (1 / (k + \text{rank}_i))$** for each method *i*, where *k* is a constant (typically 60). Documents that rank highly in multiple methods receive high RRF scores. RRF is parameter-light and robust — it consistently

outperforms methods that require careful weight tuning between retrieval signals.

Linear Score Combination

An alternative to RRF: normalize scores from each method to [0, 1] and combine with a weighted sum: **hybrid_score = alpha * bm25_score + (1 - alpha) * vector_score**. Requires tuning alpha on a validation set, but allows domain-specific weighting.

6.4 Metadata Filtering

Metadata filtering restricts the search space before or after vector search, enabling powerful structured constraints on retrieval.

- **Pre-filtering (filter then search):** Apply metadata filters first to create a subset, then search within it. Fast but can hurt recall if the subset is small.
- **Post-filtering (search then filter):** Retrieve top-K semantically, then filter by metadata. Better recall but may return fewer than K results.
- **Composite filtering:** Vector databases like Qdrant support filtering that runs in parallel with HNSW traversal, avoiding the pre/post filter tradeoff.

Example Metadata Filters

- **Date range:** retrieve only documents published after 2023-01-01.
- **Department:** retrieve only HR policy documents.
- **Source:** retrieve only from verified, high-quality sources.
- **Language:** retrieve only English-language documents.

6.5 Advanced Retrieval Strategies

Query Expansion

Enrich the query with additional related terms before retrieval. Can use an LLM to generate query variants, extract key entities, or identify synonyms. Multiple expanded queries are run and results merged.

HyDE (Hypothetical Document Embeddings)

HyDE addresses the query-document embedding gap: queries are often short questions while documents are long answers, and these live in different parts of the embedding space. HyDE uses an LLM to generate a **hypothetical answer** to the query, embeds this hypothetical document (which looks more like real documents), and uses this embedding for retrieval. This can significantly improve recall for question-answering tasks.

Multi-Query Retrieval

Use an LLM to generate 3-5 different phrasings of the user's query. Run retrieval for each phrasing and merge results. Reduces sensitivity to query phrasing and increases overall recall at the cost of higher retrieval latency and cost.

Step-Back Prompting

For queries that require general background knowledge, first ask a 'step-back' question (a more general version) to retrieve foundational context, then proceed with the original specific query. Particularly effective for multi-hop reasoning.

FEYNMAN TECHNIQUE — Hybrid Retrieval with RRF

Explain it Simply: Hybrid retrieval is like using two different librarians simultaneously — one who is excellent at exact keyword matching (a classical librarian who checks the index) and one who understands the big picture (a modern librarian who understands meaning). RRF combines their recommendations: if both agree a book is relevant, it shoots to the top. If only one recommends it, it might still be worth reading.

Real-World Analogy: Imagine you're searching for 'heart attack treatment'. The keyword librarian finds every book containing exactly those words. The semantic librarian finds books about 'myocardial infarction management', 'cardiac emergency protocols', and 'coronary care' — books the first librarian missed. Hybrid retrieval with RRF merges their lists, ranking highest the books both recommend.

Concrete Example: Query: 'Python async best practices'. BM25 retrieves documents with exact words 'Python', 'async', 'best practices'. Vector search retrieves documents about 'asyncio patterns', 'concurrency in Python', 'non-blocking I/O Python'. The merged RRF list contains the most relevant documents from both — higher recall than either method alone. Benchmarks consistently show hybrid retrieval improves recall@10 by 5-15% over either single method.

Key Takeaways — Chapter 6

- Keyword (BM25) excels at exact matches; semantic search excels at conceptual similarity.
- Hybrid retrieval combining BM25 + vector search consistently outperforms either alone.
- RRF is the go-to rank fusion algorithm: simple, robust, and parameter-light.
- Metadata filtering adds structured constraints, dramatically narrowing irrelevant results.
- Advanced techniques (HyDE, multi-query, query expansion) improve recall for complex queries.

Retrieval Evaluation

You cannot improve what you cannot measure. Retrieval evaluation gives you the quantitative foundation to make principled decisions about chunking strategies, embedding models, retrieval algorithms, and system configuration. This chapter covers the core metrics used to evaluate information retrieval systems, from classical IR measures to modern RAG-specific evaluation frameworks.

7.1 Precision and Recall

Precision and Recall are the foundational metrics of any retrieval system. They measure different aspects of retrieval quality and are often in tension with each other.

- **Precision:** Of all retrieved documents, what fraction are actually relevant? $\text{Precision} = \frac{|\text{Relevant Retrieved}|}{|\text{Retrieved}|}$
- **Recall:** Of all relevant documents, what fraction were successfully retrieved? $\text{Recall} = \frac{|\text{Relevant Retrieved}|}{|\text{Total Relevant}|}$

A system can achieve high precision by retrieving very few documents (only retrieving what it is most confident about). It can achieve high recall by retrieving many documents (casting a wide net). The **F1 score** is the harmonic mean of precision and recall, balancing both: $F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$.

In RAG contexts, recall is typically more important than precision. A missed relevant document means the LLM lacks the information needed to answer correctly. A slightly irrelevant document in the context is usually harmless — the LLM can ignore it. This motivates retrieving top-10 or top-20 documents (high recall) and relying on the LLM's attention mechanism to focus on what's relevant.

7.2 Recall@K

Recall@K measures what fraction of all relevant documents for a query appear in the top-K retrieved results. For RAG systems with a fixed context window, K is typically 5, 10, or 20. $\text{Recall@10} = 0.8$ means that on average, 80% of the relevant documents for a query are found in the top 10 results.

Recall@K is the primary metric for RAG retrieval evaluation because it directly models the realistic scenario: you retrieve K documents, pass them to the LLM, and need the answer to be within those K documents. If it's not, no amount of prompt engineering will fix the problem.

7.3 MRR (Mean Reciprocal Rank)

MRR measures how highly the first relevant result is ranked, averaged over all queries. For each query, the Reciprocal Rank (RR) is $1/\text{rank}$ of the first relevant document. If the first relevant

document is at rank 1, $RR = 1.0$. If at rank 2, $RR = 0.5$. If at rank 3, $RR = 0.33$. MRR is the mean of RR across all queries.

MRR is most meaningful when you only care about the single best answer (like a question-answering system returning one answer). For RAG systems that use multiple retrieved documents, MAP and NDCG are more informative.

7.4 MAP (Mean Average Precision)

MAP is the mean, over all queries, of the Average Precision (AP) for each query. AP for a single query computes $Precision@K$ for each rank K where a relevant document appears, then averages these values. AP rewards systems that rank relevant documents highly — a system that finds all relevant documents but ranks them at positions 50, 51, 52 will have a very low AP, even though recall is perfect.

MAP is the standard evaluation metric for information retrieval research and the metric most directly correlated with user satisfaction in search engine contexts.

7.5 NDCG (Normalized Discounted Cumulative Gain)

NDCG extends beyond binary relevance (relevant/not relevant) to handle graded relevance (e.g., highly relevant, somewhat relevant, not relevant). It computes a gain for each result position, discounted logarithmically by rank position (earlier positions matter more). The result is normalized against the ideal ranking ($DCG@K / \text{ideal } DCG@K$), producing a score in $[0, 1]$.

$NDCG@K$ is the metric of choice for modern retrieval evaluation because it simultaneously rewards: (1) finding all relevant documents, (2) ranking more relevant documents higher, and (3) placing results in an appropriate graded order.

Metric	What It Measures	Range	When to Use
Precision@K	Fraction of top-K that are relevant	$[0, 1]$	When you care about result quality
Recall@K	Fraction of all relevant found in top-K	$[0, 1]$	RAG: primary retrieval metric
MRR	Rank of first relevant result	$[0, 1]$	Single-answer QA systems
MAP	Average precision across all queries	$[0, 1]$	Multi-document retrieval
NDCG@K	Ranked retrieval with graded relevance	$[0, 1]$	Complex ranking evaluation

Table 7.1: Retrieval evaluation metrics summary

7.6 Offline vs Online Evaluation

Offline Evaluation

Offline evaluation uses a pre-built labeled dataset of queries with known relevant documents (a **golden test set**). Metrics like Recall@K, MRR, and NDCG are computed on this fixed dataset, enabling reproducible comparisons between systems.

- **Synthetic test sets:** Use an LLM to generate questions from your document corpus. Fast and cheap, but may not reflect real user queries.
- **Human-labeled test sets:** Domain experts label query-document relevance. Gold standard but expensive.
- **BEIR benchmark:** A standard multi-domain IR benchmark for comparing retrieval systems.

Online Evaluation

Online evaluation measures retrieval quality on real user queries in production. Techniques include A/B testing different retrieval configurations, tracking user feedback signals (thumbs up/down, follow-up questions), and monitoring upstream generation quality as a proxy for retrieval quality.

FEYNMAN TECHNIQUE — Retrieval Evaluation Metrics

Explain it Simply: Evaluating a retrieval system is like grading a student's search results — you give them a question, they return a list of answers, and you check: did they find the right answers? Did they find them early in their list? Did they find all of them? Different metrics measure different aspects of this 'grade'.

Real-World Analogy: Imagine you're a teacher asking students to find relevant books for an essay. Precision asks: 'Of the 5 books they brought, how many were actually relevant?' Recall asks: 'Of all the relevant books in the library, how many did they find?' MRR asks: 'How quickly did they find the FIRST relevant book?' NDCG asks: 'Did they rank the BEST books first, not just find them somewhere in the list?'

Concrete Example: Query: 'How do I reset my password?'. The knowledge base has 3 relevant documents. System returns [doc_A(relevant), doc_B(irrelevant), doc_C(relevant), doc_D(irrelevant), doc_E(relevant)]. Precision@5 = $3/5 = 0.6$. Recall@5 = $3/3 = 1.0$. MRR = $1/1 = 1.0$ (first result is relevant). MAP = $\text{mean}(1/1, 2/3, 3/5) = \text{mean}(1.0, 0.667, 0.6) = 0.756$. All metrics together give a complete picture.

Key Takeaways — Chapter 7

- Recall@K is the primary metric for RAG retrieval — it directly models whether the answer is accessible to the LLM.

- MRR is best for single-answer QA; MAP and NDCG are better for multi-document retrieval scenarios.
- Always build a domain-specific golden test set — generic benchmarks may not reflect your use case.
- Offline metrics on labeled test sets complement online A/B testing and user feedback signals.
- A retrieval system with high Recall@10 (>0.8) is a good baseline before optimizing generation.

Re-ranking

First-stage retrieval — whether keyword, semantic, or hybrid — is designed to be fast. Speed requires compromises: approximate algorithms, independent encoding, and limited context. Re-ranking is the second stage that applies a more powerful, computationally expensive relevance model to a small candidate set retrieved by the first stage. It is one of the highest-ROI optimizations in a RAG pipeline.

8.1 Why Re-ranking Is Necessary

First-stage retrieval systems encode queries and documents independently — the query is embedded without seeing the document content and vice versa. This independence is essential for scalability (document embeddings can be pre-computed) but limits relevance quality. The model cannot consider the specific interaction between the query terms and the document content.

Consider a query: 'What are the side effects of aspirin during pregnancy?' A bi-encoder embeds this as a single vector and retrieves documents about aspirin. But a document about 'aspirin and blood thinning' and a document about 'aspirin safety during first trimester' may have similar embeddings to the query, while only the second is truly relevant. A re-ranker can read both the query AND the document together, capturing this nuance.

8.2 Multi-Stage Retrieval Pipelines

The standard RAG retrieval pipeline is a **multi-stage funnel**:

- **Stage 1 (Recall-focused):** Fast retrieval of top-100 or top-500 candidates using BM25, vector search, or hybrid.
- **Stage 2 (Precision-focused):** Re-rank the candidates using a powerful cross-encoder model; keep top-5 to top-10.
- **Stage 3 (Context selection):** Optionally apply additional filters (recency, authority, diversity) to the re-ranked list.

This funnel approach achieves the best of both worlds: the fast first-stage casts a wide net (high recall), and the expensive second-stage surgically selects the most relevant documents (high precision), all within an acceptable total latency budget (typically < 100ms).

8.3 Bi-Encoder vs Cross-Encoder

Bi-Encoder (First-Stage Retriever)

Independently encodes the query and each document into vectors. Similarity is computed via a simple dot product or cosine similarity. **Key advantage:** document embeddings are pre-computed

offline, so query time only requires one forward pass (for the query) plus fast ANN search. Scales to billions of documents. **Key limitation:** cannot model fine-grained query-document interactions.

Cross-Encoder (Re-ranker)

Takes the query AND a document as a single concatenated input: '[CLS] query [SEP] document [SEP]'. The model produces a single relevance score. Because the model sees both inputs simultaneously, its self-attention layers can capture direct query-document interactions. **Key advantage:** Much higher relevance accuracy. **Key limitation:** Cannot pre-compute scores; must run a full forward pass for each (query, document) pair at query time. Scales only to 50-200 documents per query, not millions.

Property	Bi-Encoder	Cross-Encoder (Re-ranker)
Input	Query and doc separately	Query + doc concatenated
Interaction	Only via dot product	Full attention between query and doc
Latency	Sub-millisecond	50-500ms for 100 docs
Scalability	Millions/billions of docs	50-200 docs (requires first-stage)
Accuracy	Good (ANN approximate)	Excellent (full cross-attention)
Pre-computation	Documents pre-embeddable	Must run at query time

Table 8.1: Bi-encoder vs Cross-encoder comparison

8.4 Popular Re-ranking Models

- **Cohere Rerank v3:** State-of-the-art commercial re-ranker via API; strong multilingual support.
- **BGE Reranker (BAAI):** Open-source, high quality, multiple sizes (base, large).
- **ms-marco-MiniLM-L6-v2:** Fast, lightweight, good baseline; trained on MS MARCO passage retrieval.
- **JinaAI Reranker v2:** Open-source with strong performance; good for long documents.
- **MonoT5:** T5-based re-ranker using 'true/false' generation as the relevance signal.
- **LLM-as-Reranker:** Use a small LLM (e.g., GPT-4o-mini) to directly score document relevance; highest quality but slowest.

8.5 Improving Retrieval Quality

Re-ranking is one of several techniques to improve end-to-end retrieval quality. The best systems combine multiple improvement strategies:

- **Hybrid first-stage:** Combine BM25 + dense retrieval before re-ranking.

- **Longer candidate list:** Retrieve top-100 before re-ranking to top-5 rather than retrieve top-5 directly.
- **Domain-adapted re-ranker:** Fine-tune a re-ranker on domain-specific (query, relevant_doc, irrelevant_doc) triplets.
- **Diversity re-ranking:** Apply Maximal Marginal Relevance (MMR) to balance relevance with diversity, reducing redundancy in the final context.

FEYNMAN TECHNIQUE — Re-ranking in Multi-Stage Retrieval

Explain it Simply: Re-ranking is like a hiring process with two rounds. In the first round (first-stage retrieval), HR quickly screens 500 resumes and picks 20 promising candidates — fast, imperfect. In the second round (re-ranking), the hiring manager does a detailed 30-minute interview with each of the 20 candidates and ranks them precisely. The two-stage process gets you the best of both worlds: speed from the first stage, precision from the second.

Real-World Analogy: Think of it like a cooking competition. The first stage is a quick taste test — every dish gets a quick try, and the bottom 80% are eliminated. The re-ranking stage is where the judges do a detailed evaluation of the remaining dishes — texture, balance, creativity, presentation. The final ranking from the detailed evaluation is far more accurate than the quick taste.

Concrete Example: Query: 'treatment for type 2 diabetes in elderly patients with kidney disease'. First-stage retrieves 100 documents about diabetes, kidney disease, elderly care, and treatments. A bi-encoder scores these purely on embedding similarity — it might rank a general diabetes document above a highly specific elderly + kidney complication document. The cross-encoder re-ranker reads the full query AND each document together, correctly identifying that the specific combination of elderly + kidney disease + diabetes treatment is what makes a document most relevant.

Key Takeaways — Chapter 8

- Re-ranking is a second-stage precision step applied to a small candidate set from first-stage retrieval.
- Cross-encoders dramatically outperform bi-encoders for relevance scoring by modeling query-document interactions.
- The multi-stage funnel (retrieve 100, re-rank to 10) achieves high recall AND high precision.
- Commercial options (Cohere Rerank) and open-source options (BGE Reranker) both perform well.
- Re-ranking is often the highest-ROI optimization step after getting basic RAG working.

Augmentation

Augmentation is the bridge between retrieval and generation. Once relevant documents have been retrieved, they must be formatted, structured, and injected into the LLM's prompt in a way that maximizes the model's ability to use them effectively. Good augmentation dramatically improves generation quality; poor augmentation wastes the retrieval effort entirely.

9.1 Context Construction

Context construction is the process of transforming retrieved document chunks into a well-structured prompt context that the LLM can reason over. The raw retrieved text is rarely suitable for direct injection — it needs to be formatted, labeled, and ordered to optimize the LLM's comprehension.

Context Formatting Strategies

- **Simple concatenation:** Join retrieved chunks with newlines or separators. Fast but loses source attribution.
- **Labeled sources:** Prefix each chunk with [Source 1], [Source 2], etc., enabling the LLM to reference specific sources.
- **XML/JSON structure:** Wrap chunks in structured tags: `<text>`. Helps the LLM parse multiple sources.
- **Relevance ordering:** Place most relevant chunks first (recency/relevance bias in attention).
- **Summary + full text:** For long chunks, prepend an LLM-generated summary to help the model identify key points quickly.

Context Window Budget Management

The context window must accommodate the system prompt, user query, retrieved context, and generated output tokens. Context budget allocation:

- **System prompt:** 200-800 tokens for instructions, persona, and constraints.
- **Retrieved context:** Largest allocation; typically 2,000-8,000 tokens for 5-10 chunks.
- **User query:** 50-500 tokens.
- **Output tokens:** Reserve 512-2,048 for the generated response.

9.2 Prompt Augmentation

A well-designed prompt template is crucial for making the LLM use the retrieved context correctly. Key components:

- **Instruction:** Explicitly tell the model to use the provided context ('Answer based on the following documents').
- **Grounding instruction:** Tell the model to only use information from the context ('Do not use knowledge outside the provided documents').
- **Uncertainty handling:** Instruct the model on what to do when the answer isn't in the context ('If the answer is not in the documents, say so').
- **Citation instruction:** Ask the model to cite which source it used ('Cite the document source for each claim').
- **Format instruction:** Specify the desired output format (bullet points, JSON, numbered steps, etc.).

Example RAG Prompt Template

You are a helpful assistant. Answer the user's question based ONLY on the provided documents. If the answer is not found in the documents, say 'I don't have enough information to answer this question.' Cite the source document number for each claim you make.

Documents: [CONTEXT_PLACEHOLDER]

Question: [QUERY_PLACEHOLDER]

Answer:

9.3 Multi-Document Augmentation

When multiple documents are retrieved, the augmentation strategy must handle redundancy (multiple chunks saying the same thing), contradiction (different chunks asserting conflicting facts), and complementarity (chunks that together answer the query, but no single chunk suffices alone).

- **Redundancy handling:** Apply MMR (Maximal Marginal Relevance) during selection to reduce duplicate content.
- **Contradiction handling:** Include a prompt instruction: 'If documents contradict each other, note the disagreement in your answer.'
- **Long-context ordering:** Research shows LLMs perform best when relevant docs are at the beginning or end of the context ('lost in the middle' effect). Place most relevant chunks first.

9.4 Multi-Modal RAG

Modern RAG systems increasingly handle non-textual content. Multi-modal augmentation enriches the context with information from images, tables, charts, and other formats.

- **Image understanding:** Use Vision Language Models (VLMs) to generate text descriptions of images, then index and retrieve these descriptions.
- **Table augmentation:** Convert tables to structured text (Markdown, JSON) or use specialized table understanding models.
- **Chart/figure captioning:** Use VLMs (GPT-4V, Claude 3) to generate detailed captions for charts and figures.

- **Native multi-modal retrieval:** Models like CLIP embed images and text in the same space, enabling cross-modal retrieval (find images similar to a text query).

9.5 External Knowledge Integration

Beyond static document corpora, augmentation can draw from dynamic, external knowledge sources:

- **Knowledge graphs:** Integrate graph-based knowledge (e.g., Wikidata, domain ontologies) to provide structured facts alongside document chunks.
- **Real-time data:** Fetch live data from APIs (stock prices, weather, news) and include in the context for time-sensitive queries.
- **SQL databases:** Convert natural language queries to SQL using Text-to-SQL, execute, and include results in context.
- **Code repositories:** Retrieve relevant code snippets, documentation, and commit history for developer assistance tools.

FEYNMAN TECHNIQUE — Context Augmentation

Explain it Simply: Augmentation is like giving a student the relevant pages from their textbook during an exam, with helpful annotations. It's not enough to just give them the book (too much information); you need to find the right pages, mark the key paragraphs, tell them what question each part answers, and perhaps add a note like 'look especially at the third paragraph on page 47.'

Real-World Analogy: Imagine preparing a briefing for an executive before a meeting. You don't hand them a library — you select the 3-5 most relevant reports, highlight the key facts in each, organize them by topic, and write a cover note explaining how they relate to the upcoming discussion. Good augmentation does the same for the LLM: select, structure, and frame the information.

Concrete Example: User asks: 'What is our company's refund policy for annual subscriptions?' Your retriever finds 3 relevant chunks: (1) a chunk about monthly subscriptions, (2) a chunk specifically about annual subscription refunds, and (3) an outdated policy doc. Good augmentation: label each chunk, put the most relevant (chunk 2) first, and include an instruction to only use the most recent policy. Bad augmentation: dump all three chunks together without order or labels, confusing the LLM about which policy applies.

Key Takeaways — Chapter 9

- Context construction must format, label, and order retrieved chunks for LLM comprehension.
- Prompt templates should explicitly instruct grounding, uncertainty handling, and citation.

- The 'lost in the middle' effect: place most relevant content at context start or end.
- Multi-document augmentation must handle redundancy, contradiction, and complementarity.
- Multi-modal RAG uses VLMs to convert images/tables/charts into text for retrieval and augmentation.

Generation

Generation is the final step in the RAG pipeline — the LLM reads the augmented prompt and produces a response. Understanding how LLMs generate text, what parameters control the generation process, and how to engineer prompts for RAG-specific requirements is essential for building high-quality RAG systems.

10.1 How LLMs Generate Tokens

Modern LLMs are **autoregressive language models**: they generate text one token at a time, where each new token is conditioned on all previous tokens in the sequence (including the input prompt). This sequential process continues until a stop condition is met (end-of-sequence token, maximum token limit, or a specified stop string).

At each generation step, the transformer processes the current sequence through its attention layers and produces a probability distribution over the entire vocabulary (typically 32,000 to 128,000 tokens). The model then **samples** from this distribution (or selects the highest-probability token) to produce the next token. This sample is appended to the sequence and the process repeats.

10.2 Token Prediction Process

The final layer of the transformer produces a vector (the **logit vector**) of the same dimension as the vocabulary. The softmax function transforms these logits into a probability distribution. Generation parameters manipulate this distribution before sampling to control the creativity, focus, and repetition of the output.

10.3 Generation Parameters

Temperature

Temperature controls the sharpness of the probability distribution. A temperature of 1.0 uses the raw softmax probabilities. Temperature > 1.0 flattens the distribution (makes unlikely tokens more probable = more creative/random). Temperature < 1.0 sharpens the distribution (makes the most probable token even more dominant = more focused/deterministic). Temperature = 0 always selects the highest-probability token (greedy decoding).

- **Low temperature (0.0-0.3)**: Factual, deterministic responses. Best for RAG, Q&A, data extraction.
- **Medium temperature (0.5-0.7)**: Balanced creativity and coherence. Good for summarization.
- **High temperature (0.8-1.0)**: Creative, diverse outputs. Best for creative writing, brainstorming.

Top-K Sampling

Top-K sampling restricts sampling to only the K most probable tokens at each step. If K=50, only the top 50 most likely tokens can be sampled (others have their probability set to 0 before renormalization). This prevents extremely unlikely tokens from being selected, improving output quality. Common values: K=20 to K=100.

Top-P (Nucleus) Sampling

Top-P (nucleus) sampling is an adaptive alternative to Top-K. Instead of a fixed K, it selects the smallest set of tokens whose cumulative probability exceeds threshold P. If P=0.9, the model samples from the minimal set of tokens that together account for 90% of the probability mass. This dynamically adjusts the candidate pool based on the probability distribution — using fewer candidates when the model is confident, more when it is uncertain.

Frequency Penalty

Frequency penalty reduces the probability of tokens in proportion to how many times they have already appeared in the output. Discourages repetitive text. Typically in range [0, 2]; values > 0.5 noticeably reduce repetition.

Presence Penalty

Presence penalty reduces the probability of tokens that have appeared at all (regardless of how many times), encouraging the model to use a broader vocabulary. More aggressive than frequency penalty at introducing topic diversity.

Parameter	Range	RAG Recommendation	Effect
Temperature	0.0-2.0	0.0-0.3	Lower = more factual, higher = more creative
Top-K	1-500	40-80	Restricts to K most probable tokens
Top-P	0.0-1.0	0.9-0.95	Samples from top-P probability mass
Freq. Penalty	0.0-2.0	0.1-0.3	Reduces repetition of frequent tokens
Presence Penalty	0.0-2.0	0.0-0.1	Encourages vocabulary diversity
Max Tokens	1-128K	500-2000	Maximum length of generated response

Table 10.1: Generation parameters and RAG recommendations

10.4 Hallucination Mitigation

Even with retrieved context, LLMs can still hallucinate. Key mitigation strategies:

- **Grounding instructions:** Explicitly instruct the model to only use provided context.
- **Low temperature:** Temperature 0.0-0.2 reduces creative fabrication.
- **Citation requirement:** Require the model to cite source documents; uncitable claims are flagged.
- **Chain-of-thought verification:** Ask the model to first quote the relevant passage, then answer.
- **Post-processing faithfulness check:** Use a second LLM call to verify each claim in the response against retrieved documents.

10.5 Prompt Engineering for RAG

Effective RAG prompts share common patterns:

- **Clear role definition:** 'You are a precise, helpful assistant that answers questions based only on provided documents.'
- **Explicit grounding:** 'Use ONLY the information in the documents below. Do not use prior knowledge.'
- **Structured context delimiter:** Use XML tags or clear separators to delineate context from query.
- **Uncertainty handling:** 'If the documents do not contain enough information, respond: I cannot find this information in the provided sources.'
- **Output format specification:** 'Provide a concise answer in 2-3 sentences, followed by the source document numbers used.'

FEYNMAN TECHNIQUE — Autoregressive Token Generation

Explain it Simply: An LLM generates text like a very sophisticated predictive keyboard. At each step, it looks at everything written so far (including your prompt) and predicts the single most likely next word (or part of a word). Then it does it again. And again. Temperature controls how adventurous it is with these predictions — low temperature makes it pick the safest, most expected word; high temperature makes it more willing to pick surprising alternatives.

Real-World Analogy: Think of a jazz musician improvising. At low temperature, they play the 'safe' note that fits the chord perfectly — predictable but always correct. At high temperature, they experiment with unexpected notes — sometimes brilliant, sometimes clunky. For a RAG system answering factual questions, you want the safe player — low temperature, stick to what the context documents say, no risky improvisation.

Concrete Example: You ask a RAG system: 'When was the company founded?' with a document in context that says 'Founded in 1987'. At temperature=0, the model reads '1987' in the context and generates 'The company was founded in 1987.' — deterministic, correct. At temperature=1.0, the model might still say '1987', but could occasionally generate '1978' or '1997' — similar-looking numbers that seem plausible. For RAG, always use low temperature for factual queries.

Key Takeaways — Chapter 10

- LLMs are autoregressive: they generate one token at a time, each conditioned on all prior tokens.
- Temperature is the most impactful parameter for RAG: use 0.0-0.3 for factual, grounded responses.
- Top-P (nucleus sampling) is preferred over Top-K for its adaptive sampling behavior.
- Grounding instructions, citation requirements, and low temperature together minimize hallucination.
- Good RAG prompts explicitly define the LLM's role, context format, and uncertainty handling protocol.

Generation Evaluation

Evaluating the output of a RAG generation system is fundamentally different from evaluating retrieval. We are no longer asking 'did we find the right documents?' but rather 'given those documents, did the model produce an accurate, relevant, and well-grounded response?' This chapter covers the key metrics and frameworks for holistic RAG system evaluation.

11.1 Faithfulness

Faithfulness measures whether every claim in the generated answer is supported by the retrieved context documents. An answer is faithful if it contains no information that contradicts or goes beyond what the provided documents say. Faithfulness directly measures hallucination at the generation stage.

Evaluating faithfulness: decompose the generated answer into individual factual claims, then check each claim against the retrieved context. A claim is faithful if it is directly entailed by or consistent with the context. The faithfulness score is the fraction of claims that pass this check.

- **High faithfulness:** Every claim in the answer is verifiable in the context. Score: > 0.9.
- **Low faithfulness:** The model fabricated facts, extrapolated beyond context, or contradicted sources.

11.2 Answer Relevancy

Answer Relevancy measures how well the generated answer addresses the user's actual question. An answer can be perfectly faithful (fully grounded in context) but irrelevant (answering a different question). Conversely, an answer can be highly relevant but unfaithful (using relevant facts but from external memory, not the retrieved context).

Evaluating answer relevancy: reverse-engineer the question from the generated answer (using an LLM) and compute the similarity between the original question and the reverse-engineered question. High similarity = the answer actually addresses the question.

11.3 Context Utilization

Context Utilization (also called Context Precision and Context Recall) measures how well the RAG system uses the retrieved context:

- **Context Precision:** What fraction of the retrieved chunks were actually useful for generating the answer? (Measures retrieval efficiency — are we retrieving relevant chunks?)
- **Context Recall:** Were all the pieces of information needed to answer the question present in the retrieved context? (Measures retrieval completeness — did we retrieve everything needed?)

11.4 LLM-as-a-Judge

LLM-as-a-Judge uses a powerful LLM (e.g., GPT-4) as an automated evaluator to score RAG system outputs. The evaluator is given the query, the retrieved context, and the generated answer, and asked to score each of multiple dimensions on a 1-5 scale (faithfulness, relevance, completeness, coherence).

This approach scales evaluation to thousands of examples without human annotation, but introduces biases of the judge LLM. Calibration studies show that GPT-4 as judge correlates well with human judgment (Pearson $r > 0.8$) for most RAG evaluation tasks, making it the most widely used automated evaluation approach.

- **Advantages:** Scalable, nuanced, no need for ground-truth answers (reference-free).
- **Disadvantages:** Expensive, non-deterministic, biases toward its own generation style, can't evaluate very specialized domains.

11.5 End-to-End RAG Evaluation

RAGAS Framework

RAGAS (Retrieval Augmented Generation Assessment) is the most widely adopted open-source framework for end-to-end RAG evaluation. It computes four core metrics using a combination of the retrieved context, generated answer, and ground-truth answer:

- **Faithfulness:** Is every claim in the answer grounded in the context?
- **Answer Relevance:** Does the answer actually address the question?
- **Context Precision:** What fraction of retrieved chunks were relevant?
- **Context Recall:** Were all necessary facts present in the retrieved context?

TruLens

An open-source evaluation framework by TruEra, providing the **RAG Triad** evaluation: Context Relevance (is the context relevant to the query?), Groundedness (is the answer grounded in context?), and Answer Relevance (does the answer address the query?). Integrates with LangChain and LlamaIndex.

Metric	What It Measures	Requires Ground Truth?	Tool
Faithfulness	Hallucination in the answer	No	RAGAS, TruLens
Answer Relevancy	Does answer address query?	Soft (reference-free)	RAGAS, TruLens
Context Precision	Fraction of context that was useful	No	RAGAS

Context Recall	Coverage of needed info in context	Yes (ground truth answer)	RAGAS
BLEU / ROUGE	Surface-level text similarity	Yes (reference answer)	Standard NLP tools
LLM-as-Judge	Holistic quality score	No	Custom prompts, G-Eval

Table 11.1: Generation evaluation metrics summary

FEYNMAN TECHNIQUE — RAG Generation Evaluation

Explain it Simply: Evaluating a RAG response is like grading an open-book exam where you check three things: (1) Faithfulness: Did the student only write things that were in the allowed reference materials? (2) Relevance: Did they actually answer the question asked? (3) Context use: Did they find and use the right reference materials?

Real-World Analogy: Think of a lawyer's closing argument. Good evaluation checks: Did they only use admissible evidence (faithfulness)? Did their argument actually address the charges (relevance)? Did they cite all the key evidence available (context recall)? Did they avoid citing irrelevant evidence that muddled the argument (context precision)?

Concrete Example: Query: 'What is the maximum download speed of the Acme Pro plan?' Context: 'The Acme Pro plan offers download speeds up to 500 Mbps.' Generated: 'The Acme Pro plan offers 500 Mbps download speed, suitable for HD streaming.' Faithfulness: HIGH (500 Mbps is in context; 'suitable for HD streaming' is an inference — flag it). Answer Relevance: HIGH (directly answers the question). Context Precision: HIGH (the retrieved chunk was exactly what was needed).

Key Takeaways — Chapter 11

- Faithfulness, Answer Relevancy, Context Precision, and Context Recall are the four pillars of RAG evaluation.
- RAGAS is the most widely used open-source framework for automated end-to-end RAG evaluation.
- LLM-as-a-Judge enables scalable evaluation without ground-truth references but introduces judge bias.
- Always complement automated metrics with human evaluation for high-stakes production systems.
- Low faithfulness = hallucination problem; low answer relevancy = prompt/retrieval problem; low context recall = retrieval problem.

RAG vs Fine-tuning

One of the most common strategic questions in applied LLM engineering is: should I use RAG or fine-tuning to adapt the model to my domain? This chapter provides a rigorous framework for answering this question, covering when each approach is appropriate, how to combine them, and the cost, scalability, and maintenance implications of each choice.

12.1 When to Use RAG

RAG is the right choice when your primary need is **knowledge grounding** — making the model produce accurate, up-to-date, verifiable answers about specific information. RAG is preferred when:

- The knowledge base changes frequently (weekly, monthly) and retraining is impractical.
- You need citations and source attribution for transparency and auditability.
- The corpus is large (millions of documents) that won't fit in any context window.
- Different user segments need access to different knowledge bases (multi-tenant RAG).
- You want to update knowledge without model retraining (cheaper, faster iteration).
- The primary issue is factual accuracy, not writing style or task format.
- You need tight control over what information the model uses.

12.2 When to Fine-tune

Fine-tuning is the right choice when your need is **behavioral adaptation** — changing how the model responds, not just what it knows. Fine-tuning is preferred when:

- The model needs to learn a specific output format or structure not achievable through prompting.
- You need the model to follow complex domain-specific instructions consistently.
- Latency is critical and prompt engineering has become too verbose.
- The knowledge is stable and unlikely to change (historical facts, established science).
- You want to inject domain vocabulary, acronyms, or jargon into the model's 'intuition'.
- The primary issue is style, tone, or persona, not factual content.
- RAG is already implemented but generation quality (not retrieval) is the bottleneck.

Dimension	RAG	Fine-tuning
Primary strength	Dynamic, grounded knowledge	Behavioral / style adaptation
Knowledge update	Update vector DB (no retraining)	Full retraining required

Setup cost	Medium (pipeline + indexing)	High (training compute)
Ongoing cost	Storage + retrieval API	Re-training for updates
Transparency	High (citable sources)	Low (knowledge in weights)
Latency	Higher (retrieval + generation)	Lower (no retrieval step)
Hallucination risk	Low (grounded in context)	Medium-High (parametric only)
Domain adaptation	Partial (retrieval helps)	Strong (adjusts model behavior)
Best for	Dynamic knowledge, Q&A;	Style, format, behavior

Table 12.1: RAG vs Fine-tuning comparison

12.3 Combining RAG and Fine-tuning

The two approaches are not mutually exclusive — the most powerful systems often combine both. A model fine-tuned on domain-specific data will better understand domain vocabulary in retrieved documents. A RAG system using a fine-tuned model benefits from both dynamic knowledge retrieval and improved domain reasoning.

- **Fine-tune first, then RAG:** Fine-tune on domain data to improve domain understanding, then add RAG for dynamic knowledge. Most common production pattern.
- **RA-DIT (Retrieval-Augmented Dual Instruction Tuning):** Meta's approach — fine-tune the model specifically to use retrieved context effectively.
- **RAG-aware fine-tuning:** Fine-tune on (query, context, answer) triples to teach the model to properly leverage retrieved context.
- **Retriever fine-tuning:** Fine-tune the embedding model on domain-specific (query, relevant_doc) pairs to improve retrieval. Often higher ROI than fine-tuning the generator.

12.4 Cost, Scalability, and Maintenance

RAG Cost Structure

- **Setup:** Compute for embedding all documents (one-time, moderate cost).
- **Infrastructure:** Vector database hosting (\$50-\$500/month for managed services).
- **Per-query:** Embedding the query + API costs for retrieval + generation.
- **Update:** Re-embed and re-index new or changed documents (incremental, cheap).

Fine-tuning Cost Structure

- **Setup:** Training compute; depends on model size. Fine-tuning GPT-3.5 Turbo costs ~\$500-\$5,000 depending on data volume.
- **Infrastructure:** Hosting the fine-tuned model (if self-hosted).

- **Per-query:** Similar to base model; slightly lower if using smaller fine-tuned model.
- **Update:** Full retraining; must repeat the entire fine-tuning process.

FEYNMAN TECHNIQUE — RAG vs Fine-tuning Decision

Explain it Simply: Choosing between RAG and fine-tuning is like deciding whether to give a doctor a reference book or send them back to medical school. Giving them a reference book (RAG) is fast, cheap, and keeps them updated with the latest research — but they still need to look things up. Sending them back to school (fine-tuning) takes longer and costs more, but afterward they have the knowledge integrated into their intuition — and they respond faster. The best choice depends on whether your problem is 'the model doesn't know the information' (use RAG) or 'the model doesn't reason about the domain correctly' (use fine-tuning).

Real-World Analogy: RAG is like a brilliant consultant with access to your company's full document library. They don't memorize everything, but they know how to look up exactly what's needed. Fine-tuning is like training an employee for 6 months in your company's specific processes — they respond faster and more naturally, but their knowledge doesn't update until you train them again.

Concrete Example: Scenario: You're building a Q&A system for a software product that releases new versions monthly. RAG wins: just update the documentation in the vector database each release. You don't need to retrain the model. Now imagine you're building a specialized medical record summarizer that must follow a specific structured format and use precise clinical language. Fine-tuning wins: RAG won't fix the output format, but fine-tuning on thousands of examples of correctly formatted summaries will. Ideal production system: fine-tune for format + style, then add RAG for knowledge.

Key Takeaways — Chapter 12

- RAG is ideal for dynamic knowledge; fine-tuning is ideal for behavioral and style adaptation.
- The two approaches are complementary and are often combined in production systems.
- RAG is dramatically cheaper to update; fine-tuning requires full retraining for knowledge updates.
- Retriever fine-tuning (on domain-specific query-doc pairs) often yields higher ROI than generator fine-tuning.
- When in doubt, start with RAG — it's faster to implement, easier to debug, and cheaper to iterate.

RAG with Tools

Traditional RAG retrieves from static document corpora. But real-world knowledge is often locked in databases, APIs, search engines, and live systems. Tool-augmented RAG extends the retrieval concept to these dynamic, structured data sources, giving the LLM access to live, structured, and heterogeneous information through function calling and tool use.

13.1 Retrieving from APIs

Many knowledge sources are not static documents but dynamic APIs: weather services, financial data providers, news feeds, product catalogs, and customer databases. Tool-augmented RAG treats API calls as retrieval operations — the LLM decides which API to call, what parameters to use, then incorporates the API response into its answer.

- **REST APIs:** The LLM generates an API endpoint and query parameters; a tool executor makes the HTTP call and returns the JSON response.
- **GraphQL APIs:** The LLM generates a GraphQL query to retrieve precisely the fields needed, avoiding over-fetching.
- **Authenticated APIs:** Credentials are managed by the tool executor, not the LLM, maintaining security.
- **Rate limiting:** Tool executors handle rate limiting and retries transparently.

13.2 Database Retrieval

Text-to-SQL is one of the most powerful tool-augmented RAG patterns. The LLM is given a schema description and generates a SQL query to retrieve exactly the data needed to answer the user's question. The query is executed against the actual database, and the results are injected into the LLM's context.

Key challenges in Text-to-SQL:

- **Schema complexity:** LLMs must understand table relationships, foreign keys, and join semantics.
- **Ambiguous queries:** 'Show me top customers' requires the LLM to infer what 'top' means (revenue? orders? recency?).
- **SQL injection risk:** Generated SQL must be validated and sandboxed before execution.
- **NoSQL databases:** MongoDB, DynamoDB queries require different syntax; often simpler for the LLM to handle.

13.3 Search Engine Integration

For queries requiring very recent information or broad web knowledge, search engine integration provides access to information beyond any static corpus. The LLM formulates a search query, a tool executes it against a search API (SerpAPI, Google Custom Search, Bing Search), and the top results are scraped and added to the RAG context.

- **SerpAPI:** Provides structured access to Google Search results.
- **Bing Search API:** Microsoft's search API with rich snippet extraction.
- **Tavily API:** AI-optimized search API specifically designed for RAG systems.
- **SearXNG:** Open-source, self-hosted search aggregator for privacy-preserving web search.

13.4 Structured and Unstructured Data Sources

Production RAG systems must handle both unstructured (free-text documents) and structured (tabular, relational) data. A well-designed tool-augmented system routes queries to the appropriate source type:

- **Unstructured (vector search):** 'What does our policy say about remote work?' → vector database retrieval.
- **Structured (SQL/API):** 'How many customers signed up last month?' → SQL database query.
- **Hybrid:** 'Who are our top customers and what contract terms do they have?' → SQL for customer metrics + vector search for contract terms.

13.5 Tool Calling for Retrieval

Modern LLM APIs (OpenAI, Anthropic Claude, Gemini) provide native **function calling** (tool use) capabilities. The LLM declares the tools available to it, and when it decides to use a tool, it outputs a structured JSON object specifying the tool name and arguments. The application executes the tool and feeds back the result, and the LLM continues generation with this additional information.

Key advantages of native function calling over prompt engineering:

- **Reliable structured output:** Tool arguments are constrained by a JSON schema, preventing malformed calls.
- **Parallel tool calls:** Modern APIs support calling multiple tools simultaneously (e.g., search while querying a database).
- **Streaming compatibility:** Tool calls can stream intermediate results for better UX.
- **Safety controls:** The application layer validates and authorizes tool calls before execution.

Tool Type	Data Source	Best For	Key Challenge
Vector Search	Document corpus	Unstructured knowledge	Chunk quality, embedding model
Text-to-SQL	Relational DB	Structured analytics	Schema complexity, SQL safety
REST API	Live services	Real-time data	Auth, rate limiting, JSON parsing

Web Search	Internet	Current events, broad facts	Source reliability, page scraping
Knowledge Graph	Ontologies	Relational facts, entity data	Graph query complexity
Code Executor	Runtime environment	Calculations, data analysis	Sandbox security

Table 13.1: Tool types for retrieval in RAG systems

FEYNMAN TECHNIQUE — Tool-Augmented RAG

Explain it Simply: Tools in RAG are like special abilities a superhero can call on. Without tools, the LLM only has its trained knowledge and retrieved documents. With tools, it can also call a weather service, run a database query, search the web, or execute a calculator — getting live, structured, or computed information on demand.

Real-World Analogy: Think of the LLM as a brilliant research analyst. Without tools, they can only work with information already on their desk (training data + retrieved docs). With tools, they can call the database team for fresh numbers, ask an assistant to Google something, run a spreadsheet calculation, or phone an expert for a real-time quote. Each tool call is like making that phone call — getting exactly the information needed.

Concrete Example: User asks: 'What is our revenue for Q3 2024 compared to the same period last year, and what does our Q3 report say about the causes?' The LLM uses TWO tools: (1) SQL tool — `SELECT SUM(revenue) FROM sales WHERE quarter='Q3' GROUP BY year` — retrieves 2024: \$12.3M, 2023: \$10.1M (+22%). (2) Vector search — retrieves the Q3 report chunk attributing growth to 'expanded European market'. Final answer combines both: structured data + unstructured narrative.

Key Takeaways — Chapter 13

- Tool-augmented RAG extends retrieval beyond static documents to APIs, databases, and search engines.
- Text-to-SQL enables natural language querying of structured data — one of the highest-value RAG patterns.
- Native function calling (tool use) provides reliable, schema-constrained tool invocation.
- Route queries to appropriate data source types: vector search for unstructured, SQL/API for structured.

- Always validate and sandbox tool calls for security; never pass LLM-generated SQL directly to production DBs.

Agentic RAG

Agentic RAG represents the frontier of intelligent retrieval-augmented systems. Where traditional RAG follows a fixed retrieve-augment-generate pipeline, Agentic RAG gives the model the ability to plan its own retrieval strategy, decide which tools to use, reflect on intermediate results, and iterate toward a complete answer. This chapter explores the architectural patterns and real-world applications of these autonomous systems.

14.1 Evolution from RAG to Agentic RAG

Traditional RAG is a **single-pass pipeline**: one query → one retrieval → one generation. This works well for simple factual questions where the relevant information is in a single document chunk. But many real-world questions require multi-hop reasoning: 'What were the key differences between our Q1 and Q2 performance, and how do they relate to our stated strategy goals?' — this requires multiple retrievals, comparisons, and synthesis that a single-pass system cannot handle.

Agentic RAG gives the LLM **agency over the retrieval process itself**. Instead of retrieval being a fixed step, the LLM becomes an active planner that decides what to retrieve, evaluates whether the retrieved information is sufficient, and iterates until it can produce a complete, accurate answer.

14.2 Dynamic Tool Selection

A core capability of Agentic RAG is dynamic selection of the appropriate retrieval tool for each sub-question. Given a set of available tools (vector search, SQL, web search, API calls), the agent decides at runtime which tool to use for each retrieval step based on the query's nature.

- **Tool router:** A lightweight classifier or the LLM itself selects the appropriate tool based on query type.
- **Tool composition:** Complex queries may require sequential tool calls, where the output of one tool informs the next.
- **Parallel tool calls:** Independent sub-questions can be processed in parallel using multiple tools simultaneously.
- **Fallback strategies:** If the primary tool fails or returns insufficient information, the agent falls back to alternative tools.

14.3 Planning and Reasoning

ReAct (Reason + Act)

ReAct is the foundational agentic pattern for tool-using LLMs. The LLM alternates between **Thought** (reasoning about what to do next), **Action** (calling a tool), and **Observation** (processing the tool's

output). This interleaved reasoning-action loop enables the agent to plan reactively — each observation informs the next thought and action.

Plan-and-Execute

A two-phase architecture: (1) **Planner**: an LLM decomposes the query into a sequence of sub-tasks (e.g., [retrieve Q1 data, retrieve Q2 data, compare, synthesize]). (2) **Executor**: a separate agent executes each sub-task in order, with the ability to adapt the plan when sub-tasks fail or produce unexpected results.

Chain-of-Thought (CoT) Retrieval

The agent uses chain-of-thought reasoning to identify what information gaps exist, formulate retrieval queries to fill those gaps, and verify whether the retrieved information is sufficient. CoT reasoning makes the agent's retrieval strategy interpretable and debuggable.

14.4 Reflection and Iterative Retrieval

Reflection is the ability of an agent to critically evaluate its own intermediate outputs and decide whether to proceed, revise, or retry. In Agentic RAG, reflection enables **iterative retrieval**: after an initial retrieval and generation, the agent checks whether the answer is complete and accurate, and if not, performs additional retrievals to fill gaps.

Self-RAG

Self-RAG is a framework where the LLM is trained to produce special reflection tokens during generation: [Retrieve] (should I retrieve?), [IsRel] (is this context relevant?), [IsSup] (is my claim supported by context?), [IsUse] (is the response useful?). These tokens enable the model to dynamically decide when and whether to retrieve, and to self-evaluate its generated claims.

CRAG (Corrective RAG)

CRAG adds a retrieval correction step: after retrieval, a lightweight relevance evaluator scores the retrieved documents. If documents are judged as 'irrelevant', CRAG triggers a web search to find better information. If 'ambiguous', it refines the documents using knowledge extraction. This corrective mechanism significantly improves robustness on out-of-domain or difficult queries.

14.5 Multi-Agent Architectures

Complex RAG tasks benefit from **multi-agent architectures** where specialized agents handle different aspects of the task:

- **Orchestrator agent**: Receives the user query, decomposes it, assigns sub-tasks to specialist agents, and synthesizes the final answer.
- **Retrieval specialist agents**: Each agent is optimized for a specific retrieval type (semantic search, SQL, web search, knowledge graph).

- **Reasoning agent:** Performs multi-hop inference over retrieved facts from multiple agents.
- **Verification agent:** Checks the synthesized answer for faithfulness and consistency before returning it to the user.
- **Memory agent:** Manages short-term (conversation) and long-term (cross-session) memory for the agent system.

LangGraph for Agentic RAG

LangGraph is the leading framework for building stateful, multi-step agentic systems. It represents the agent's control flow as a directed graph where nodes are processing steps (LLM calls, tool calls) and edges represent transitions between steps (with conditional branching). Key advantages: first-class support for cycles (enabling iterative retrieval), persistent state management, built-in streaming, and easy human-in-the-loop integration.

FEYNMAN TECHNIQUE — Agentic RAG

Explain it Simply: Agentic RAG is like upgrading from a reference desk librarian to a full research assistant. Traditional RAG is the reference desk: you ask a question, they retrieve some books, you get an answer. Agentic RAG is a research assistant who can plan their own research strategy: 'This question needs data from three sources. Let me check the financial database, search recent news, and review the annual report. If the data doesn't add up, I'll dig deeper.'

Real-World Analogy: Imagine asking a detective to solve a case. A simple RAG detective checks one file and gives you whatever is there. An agentic detective asks: 'What do I need to know? Where can I find it? Does this evidence make sense? What am I still missing?' They interview witnesses, check databases, re-examine contradictions, and form a conclusion only when they have enough evidence.

Concrete Example: Complex query: 'Compare our product return rates for Q3 2024 vs Q3 2023, identify the top 3 reasons for returns, and assess whether our new packaging change in June 2024 had any impact.' An agentic RAG system plans: (1) SQL query for return rate data both periods, (2) vector search for customer complaint themes, (3) vector search for packaging change documentation and date, (4) reasoning step to connect the timeline. After each step, it reflects: 'Do I have enough? Is this data consistent?' — iterating until it can produce a complete, multi-faceted answer.

Key Takeaways — Chapter 14

- Agentic RAG gives the LLM agency over its own retrieval strategy — it plans, acts, observes, and reflects.
- ReAct (Reason + Act) is the foundational pattern: interleaved reasoning and tool calls.
- Reflection and iterative retrieval (Self-RAG, CRAG) dramatically improve quality on complex queries.

- Multi-agent architectures use specialized agents for different retrieval types and reasoning tasks.
- LangGraph is the leading framework for stateful, cyclical, multi-step agentic RAG systems.

Production Best Practices

Building a RAG prototype is relatively straightforward. Building a production RAG system that serves thousands of users reliably, cost-effectively, and with high quality is a serious engineering challenge. This final chapter synthesizes the lessons from all previous chapters into actionable best practices for designing, deploying, monitoring, and evolving production-grade RAG and Agentic RAG systems.

15.1 Designing Production-Ready RAG Systems

Architecture Principles

- **Separation of concerns:** Keep the indexing pipeline, retrieval service, and generation service as independent, deployable units.
- **Asynchronous indexing:** Document ingestion and indexing should be async (queue-based), decoupled from the query-serving path.
- **Stateless retrieval service:** The retrieval API should be stateless and horizontally scalable; state lives in the vector database.
- **Graceful degradation:** If vector DB is unreachable, fall back to keyword search. If re-ranker is unavailable, skip to generation.
- **Multi-environment design:** Maintain separate dev, staging, and production vector databases; test indexing changes in staging first.

System Components

- **Document ingestion worker:** Watches for new documents, parses, chunks, embeds, and upserts to vector DB.
- **Retrieval API:** Stateless REST/gRPC service; accepts query, returns ranked chunks with metadata.
- **Generation API:** Takes query + context, calls LLM, returns response with citations.
- **Orchestration layer:** Routes requests, manages rate limiting, handles fallbacks.
- **Evaluation harness:** Continuously runs golden test set evaluations against production system.

15.2 Performance Optimization

Caching Strategies

- **Query embedding cache:** Cache embeddings of common queries to avoid re-embedding.
- **Retrieval result cache:** Cache top-K results for repeated queries (e.g., using Redis with TTL).
- **Semantic cache:** Use embedding similarity to cache results for semantically similar (not just identical) queries.

- **Generation cache:** For deterministic (temperature=0) and high-traffic queries, cache generated answers.

Latency Optimization

- **Parallel retrieval:** Run BM25 and vector search in parallel; merge results asynchronously.
- **Batch embedding:** Embed multiple chunks in a single API call during indexing.
- **Streaming generation:** Stream tokens to the user as they are generated, reducing perceived latency.
- **Model selection:** Use faster, cheaper models (GPT-4o-mini) for simple queries; route complex queries to stronger models (GPT-4o).
- **HNSW parameter tuning:** Reduce ef at query time to trade 1-2% recall for 2-5x speed improvement.

15.3 Cost Optimization

RAG systems can become expensive at scale if not carefully managed. Key cost levers:

- **Embedding model selection:** Open-source models (BGE, E5) eliminate embedding API costs at higher infra cost.
- **Context window minimization:** Send only the top-3 most relevant chunks instead of top-10; reduces generation cost by up to 50%.
- **Model routing:** Route 80% of queries to cheaper models (GPT-4o-mini, Haiku) and only escalate complex queries to expensive models.
- **Batching:** Use batch APIs (OpenAI Batch API) for non-real-time workloads at 50% cost reduction.
- **Aggressive caching:** Cache common queries aggressively; many production systems see 30-50% cache hit rates.

Cost Lever	Typical Savings	Implementation Complexity
Model routing (cheap for simple queries)	40-60%	Medium
Semantic caching (similar query hits)	20-40%	Medium
Batch API for async workloads	50%	Low
Open-source embeddings (self-hosted)	80-90% on embedding	High (infra)
Context window reduction (top-3 vs top-10)	30-50% on generation	Low
Compressed retrieval (smaller chunks)	10-20%	Medium

Table 15.1: Cost optimization strategies for production RAG

15.4 Common Failure Modes

- **Retrieval failure:** The relevant document exists but is not retrieved. Root causes: poor chunking, wrong embedding model, inadequate hybrid retrieval.
- **Context stuffing:** Too many retrieved chunks overwhelm the LLM; it focuses on the wrong parts. Fix: reduce K, apply re-ranking.
- **Temporal confusion:** The system retrieves outdated versions of documents. Fix: add `creation_date` metadata and prefer recency in ranking.
- **Contradiction in context:** Multiple retrieved chunks say different things. Fix: train the LLM to flag contradictions; prefer the most recent source.
- **Prompt injection:** Malicious content in retrieved documents manipulates the LLM. Fix: sanitize retrieved content; use strict output validation.
- **Embedding drift:** The embedding model changes, making new documents incompatible with old index. Fix: version your embedding model; re-index on model change.

15.5 Monitoring and Evaluation

Key Metrics to Monitor

- **Retrieval quality:** Track Recall@K and MRR on golden test sets weekly; alert on > 5% degradation.
- **Generation quality:** Track faithfulness and answer relevancy via automated LLM-as-Judge evaluation.
- **User feedback:** Collect explicit (thumbs up/down) and implicit (session abandonment, follow-up questions) feedback signals.
- **Latency:** Track p50, p95, p99 latency for retrieval, generation, and end-to-end; set SLO targets.
- **Cost per query:** Track token consumption and API costs per query type; alert on unexpected spikes.
- **Index freshness:** Track lag between document creation and indexing completion; ensure critical documents are indexed promptly.

Observability Stack

- **LangSmith:** LangChain's tracing platform; full RAG chain observability with inputs/outputs at each step.
- **Langfuse:** Open-source LLM observability with RAG-specific metrics; excellent for production monitoring.
- **MLflow:** Experiment tracking for embedding model comparisons and RAG configuration experiments.
- **Prometheus + Grafana:** Infrastructure metrics (latency, error rates, vector DB performance).

15.6 Future Directions of Agentic AI

The RAG landscape is evolving rapidly. Key trends shaping the near-term future:

- **Agentic systems at scale:** Multi-agent RAG systems that autonomously plan complex research tasks, with human-in-the-loop oversight at key decision points.
- **Long-context + RAG hybrid:** As context windows approach 1M+ tokens, the boundary between 'retrieval' and 'in-context' blurs; future systems will dynamically decide what to retrieve vs. keep in context.
- **Multimodal RAG:** Retrieval over images, audio, video, and structured data in a unified embedding space.
- **Real-time RAG:** Continuous indexing of streaming data sources (news feeds, sensor data, social media) for real-time knowledge bases.
- **Self-improving RAG:** Systems that automatically identify retrieval failures from user feedback and trigger self-refinement of chunking strategies, embedding models, and retrieval parameters.
- **MCP (Model Context Protocol):** Anthropic's open standard enabling LLMs to connect to any data source through a universal tool interface, making RAG with tools more standardized and composable.

FEYNMAN TECHNIQUE — Production RAG Systems

Explain it Simply: Building a production RAG system is like the difference between cooking a great meal at home and running a restaurant. At home, cooking one meal is easy — you have all the time in the world and can improvise. Running a restaurant means: serving hundreds of meals simultaneously, maintaining consistent quality, minimizing food waste (cost), watching for spoiled ingredients (monitoring), recovering when the stove breaks (fault tolerance), and constantly refining recipes based on customer feedback (continuous improvement).

Real-World Analogy: Think of your vector database as the restaurant's pantry. If it goes down, service stops. Your retrieval pipeline is the kitchen — it needs to produce results in 50ms, not 50 seconds. Caching is mise en place — prep work done in advance so orders execute faster during service. Monitoring is the manager walking the floor, catching problems before customers notice. Evaluation is the weekly quality review to ensure standards are maintained.

Concrete Example: A production RAG deployment at a financial services firm: (1) 10M+ documents indexed in Qdrant with daily incremental updates. (2) Hybrid retrieval (BM25 + vector) with BGE reranker achieving Recall@5 = 0.91. (3) Semantic caching with 35% hit rate. (4) Model routing: 75% of queries to GPT-4o-mini, 25% to GPT-4o. (5) LangSmith tracing + Langfuse evaluation + weekly RAGAS scores on 500-query golden set. (6) p95 end-to-end latency: 1.8 seconds. (7) Average cost: \$0.004/query.

Key Takeaways — Chapter 15

- Production RAG requires architectural separation of ingestion, retrieval, and generation services.
- Caching (query, retrieval, semantic, generation) dramatically reduces both latency and cost.

- Model routing, batch APIs, and open-source embeddings are the highest-ROI cost optimizations.
- Monitor retrieval quality, faithfulness, latency, and cost with automated alerting.
- Common failure modes (retrieval failure, context stuffing, embedding drift) require proactive mitigation.
- The future is agentic, multimodal, real-time, and self-improving RAG systems.